Layout and data display

24D. Layout & Navigation Components

Problems: 9 (Q 24.23 – Q 24.31)

Q 24.23 Infinite Scroll List

JSX Copy

```jsx
function InfiniteList({ fetchPage }) {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);
  const [loading, setLoading] = useState(false);
  const sentinelRef = useRef(null);

  useEffect(() => {
    const observer = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting && hasMore && !loading) loadMore();
    }, { rootMargin: '200px' });
    if (sentinelRef.current) observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [hasMore, loading]);

  async function loadMore() {
    setLoading(true);
    const { data, hasNextPage } = await fetchPage(page);
    setItems(prev => [...prev, ...data]);
    setHasMore(hasNextPage);
    setPage(p => p + 1);
```

```
  return (
    <ul>
      {items.map(item => <li key={item.id}>{item.name}</li>)}
      {loading && <li className="loading">Loading ...</li>}
      {hasMore && <li ref={sentinelRef} style={{ height: 1 }} />}
      {!hasMore && <li className="end">No more items</li>}
    </ul>
  );
}
```

Edge cases: Empty initial load, network error (retry button), fast scrolling triggering multiple loads (guard with `loading` flag), duplicate items (deduplicate by ID).

Q 24.24 Virtualized List (10,000+ items)

JSX Copy

```
function VirtualList({ items, itemHeight = 50, containerHeight = 500, buff
  const [scrollTop, setScrollTop] = useState(0);

  const totalHeight = items.length * itemHeight;
  const startIndex = Math.max(0, Math.floor(scrollTop / itemHeight) - buff
  const endIndex = Math.min(items.length, Math.ceil((scrollTop + container
  const visibleItems = items.slice(startIndex, endIndex);

  return (
    <div style={{ height: containerHeight, overflow: 'auto' }}
      onScroll={(e) => setScrollTop(e.currentTarget.scrollTop)}>
      <div style={{ height: totalHeight, position: 'relative' }}>
        {visibleItems.map((item, i) => (
          <div key={item.id} style={{
            position: 'absolute', top: (startIndex + i) * itemHeight,
            height: itemHeight, width: '100%',
          }}>
            {item.name}
          </div>
        ))}
```

```
  );
}
```

In production, use `@tanstack/react-virtual`.

Q 24.25 Image Carousel / Slider

Track `currentIndex`, use `transform: translateX(-${currentIndex * 100}%)` for GPU-composited sliding. Only render current + adjacent slides (lazy rendering). Touch: track `touchstart` / `touchend` x-coordinates, swipe if delta > 50px.

Accessibility: `role="region"`, `aria-roledescription="carousel"`, each slide `role="group"` with `aria-label="Slide N of M"`. Pause auto-play on hover/focus.

Q 24.26 Drag and Drop — Sortable List

JSX Copy

```jsx
function SortableList({ items, onReorder }) {
  const [dragIndex, setDragIndex] = useState(null);

  function handleDragStart(e, index) {
    setDragIndex(index);
    e.dataTransfer.effectAllowed = 'move';
  }

  function handleDragOver(e, index) {
    e.preventDefault();
    if (dragIndex === index) return;
    const reordered = [...items];
    const [moved] = reordered.splice(dragIndex, 1);
    reordered.splice(index, 0, moved);
    onReorder(reordered);
    setDragIndex(index);
```

```
  return (
    <ul>
      {items.map((item, i) => (
        <li key={item.id} draggable onDragStart={(e) => handleDragStart(e,
          onDragOver={(e) => handleDragOver(e, i)} onDragEnd={() => setDra
          className={i === dragIndex ? 'dragging' : ''}>
          {item.name}
        </li>
      ))}
    </ul>
  );
}
```

Q 24.27 Kanban Board

Multiple columns, cards draggable between columns. Focus on: working drag-and-drop, column rendering, add card. Simple `useState` with column arrays is enough for 60 minutes.

Q 24.28 Resizable Split Pane

Two panels, draggable divider. Track mouse position on drag, set left panel width as percentage. Constraints: min/max width. `cursor: col-resize` on divider.

Q 24.29 Nested Sidebar Navigation

Recursive tree of nav items. Expand/collapse toggle. Track expanded state with `Set<string>`. `aria-expanded` on toggle buttons.

Q 24.30 Command Palette (Cmd+K)

Global keydown listener opens search modal. Filter commands by input. Keyboard nav (arrows, Enter). Group by category. Recently used at top. Fuzzy search with `includes()` or `fuse.js`.

Q 24.31 Bottom Sheet (Mobile)

Slides up from bottom. Three states: closed, half-open (peek), fully open. Drag handle to resize. Swipe down past threshold to close. `transform: translateY()` for GPU animation.

24E. Data Display Components

Problems: 7 (Q 24.32 – Q 24.38)

Q 24.32 Spreadsheet-like Editable Table

JSX Copy

```jsx
function EditableTable({ data, columns, onCellChange }) {
  const [editingCell, setEditingCell] = useState(null);
  const [editValue, setEditValue] = useState('');

  function startEdit(row, col) {
    setEditingCell({ row, col });
    setEditValue(data[row][columns[col].key]);
  }

  function commitEdit() {
    if (editingCell) {
      onCellChange(editingCell.row, columns[editingCell.col].key, editValu
      setEditingCell(null);
```

```
  function handleKeyDown(e) {
    if (e.key === 'Enter') commitEdit();
    if (e.key === 'Tab') { e.preventDefault(); commitEdit(); /* move right
    if (e.key === 'Escape') setEditingCell(null);
  }

  return (
    <table>
      <thead><tr>{columns.map(col => <th key={col.key}>{col.label}</th>)}<
      <tbody>
        {data.map((row, ri) => (
          <tr key={ri}>
            {columns.map((col, ci) => (
              <td key={col.key} onDoubleClick={() => startEdit(ri, ci)}>
                {editingCell?.row === ri && editingCell?.col === ci
                  ? <input value={editValue} onChange={e => setEditValue(e
                      onKeyDown={handleKeyDown} onBlur={commitEdit} autoFo
                  : row[col.key]}
              </td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Q 24.33 Sortable and Filterable Table

Click column header to sort (asc → desc → none). Filter input above table. `useMemo` for sorted + filtered data.

Q 24.34 Tree View / File Explorer

```jsx
function TreeNode({ node, level = 0 }) {
  const [expanded, setExpanded] = useState(false);
  const hasChildren = node.children?.length > 0;

  return (
    <div>
      <div style={{ paddingLeft: level * 20 }} className="tree-node">
        {hasChildren
          ? <button onClick={() => setExpanded(!expanded)} aria-expanded={
              {expanded ? '▾' : '▸'}
            </button>
          : <span style={{ width: 20 }} />}
        <span>{hasChildren ? '📁' : '📄'} {node.name}</span>
      </div>
      {expanded && node.children?.map(child => (
        <TreeNode key={child.id} node={child} level={level + 1} />
      ))}
    </div>
  );
}
```

Q 24.35 Nested Comments

Recursive component like TreeView but with reply functionality. Depth limiting (show "View more replies" after level 3).

Q 24.36 Timeline / Activity Feed

Vertical line with events. Each has icon, timestamp, description. CSS `::before` pseudo-element for the vertical line.

CSS approach: `column-count` + `break-inside: avoid`. JS approach: calculate column heights, place items in shortest column.

Q 24.38 Responsive Data Grid

Table on desktop, card layout on mobile. Switch between `<table>` and card rendering at breakpoint.

QUICK CHECK — no marks, just for you.

1. In the InfiniteList solution, the IntersectionObserver is created with the option { rootMargin: '200px' }. What does that option achieve?

- ◯ It sets the sentinel's CSS margin to 200px so it never overlaps the last rendered list item
- ◯ The sentinel is treated as intersecting 200px before it actually reaches the viewport, so the next page starts loading early
- ◯ It requires the sentinel to be at least 200px tall before the observer will fire a callback
- ◯ It throttles the observer so its callback can fire at most once per 200 pixels of scrolling

Check answer

2. In the VirtualList solution, why is `buffer` subtracted from the start index and added to the end index?

- ◯ It compensates for the container's padding so the first item is not clipped at the top edge
- ◯ It guarantees each item's React key stays stable as the visible window slides

It reserves space for the absolutely positioned spacer div that establishes the total scroll height

- ◯ It renders a few extra rows just outside the viewport so fast scrolling does not expose blank gaps

Check answer

3. In the EditableTable solution, a user double-clicks a cell, types a new value, and then presses Escape. What happens?

- ◯ The typed value is committed via onCellChange, exactly as if Enter had been pressed
- ◯ The input stays open with its text cleared, waiting for a fresh value
- ◯ Editing is cancelled and onCellChange is never called, so the original value stays
- ◯ The typed value is committed and edit mode moves to the cell on the right

Check answer

‹ Previous
Form and input components

Next ›
Applications and advanced

LEARN MORE

ARTICLES & DOCS

TanStack Table
github.com

VIDEOS



Data table grid with sorting & filter

Frontend System Desi...



Create a React data table component

YouTube



Build a table component in React

YouTube

Hand-picked references for this lesson.